

Beginner Friendly Booklet

JavaScript Core Concepts

DOM Access • Dynamic Language • OOP • Event Loop



Simple explanations, clear visuals, readable code, and interview revision notes.

How to use this booklet

Recommended learning order

1. **DOM first** - learn how JavaScript touches the webpage.
2. **Dynamic language** - learn how values and types work.
3. **OOP** - learn how to organize code using objects and classes.
4. **Event loop** - learn why asynchronous code runs later.

Beginner mindset

Do not start by memorizing syntax. First understand the flow.

User action → JavaScript runs → DOM updates → Page changes

After that, practice small examples and predict the output before running the code.

Table of contents

1. DOM access and page data

2. Dynamic programming language

3. Object-oriented JavaScript

4. Event loop in detail

5. Mini project

6. Quick revision and interview notes

1. How JavaScript uses DOM to access data

Simple meaning

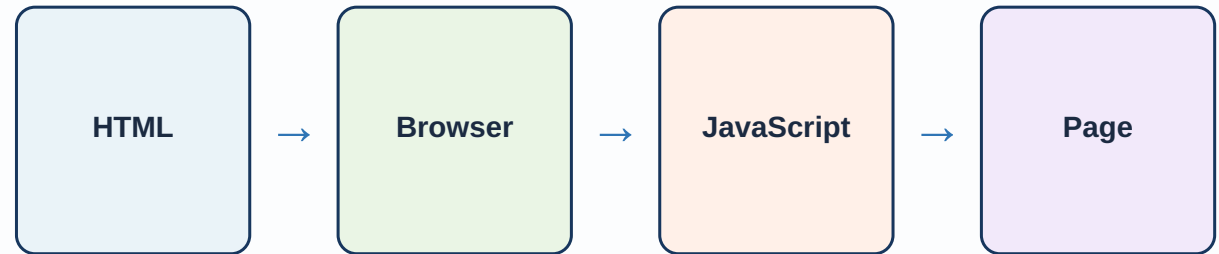
The **DOM** is the browser's object version of your HTML page.

JavaScript uses the DOM to:

- select HTML elements
- read text and input values
- change text, styles, and attributes
- add or remove elements
- react to user events like clicks and typing

DOM is the bridge between the webpage and JavaScript code.

DOM access flow



```
const value = document.getElementById("name").value;
```

JavaScript selects the DOM object and reads what the user typed.

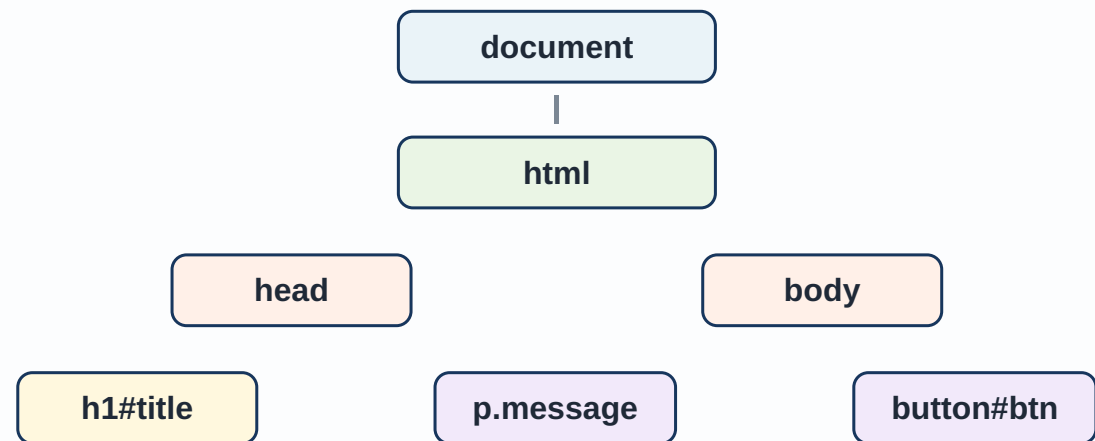
1.1 DOM tree: HTML becomes objects

Example HTML

```
<body>
  <h1 id="title">Welcome</h1>
  <p class="message">Hello</p>
  <button id="btn">Click</button>
</body>
```

After loading the page, the browser converts this HTML into a tree of objects.

DOM tree visual



Each HTML tag becomes an object that JavaScript can select and modify.

1.2 Selecting and reading DOM data

Select elements

```
const title = document.getElementById("title");
const message = document.querySelector(".message");
const items = document.querySelectorAll(".item");
```

Method	Purpose
getElementById()	Select one element by id
querySelector()	Select first matching CSS selector
querySelectorAll()	Select all matching elements

Read data

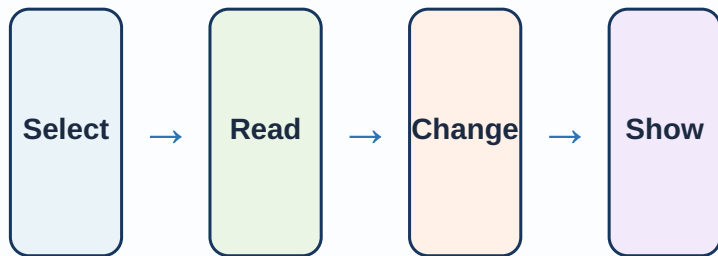
```
const text = title.innerText;
const html = box.innerHTML;
const name = input.value;
const src = image.getAttribute("src");
```

Property	Use
innerText	Read visible text
innerHTML	Read HTML content
value	Read input value
getAttribute()	Read attribute value

Important: for form fields like input, textarea, and select, use `.value`.

1.3 Updating the visible page

Most DOM work follows this pattern



innerText innerHTML value style appendChild()

Common DOM updates

```
// Change text
title.innerText = "New Title";

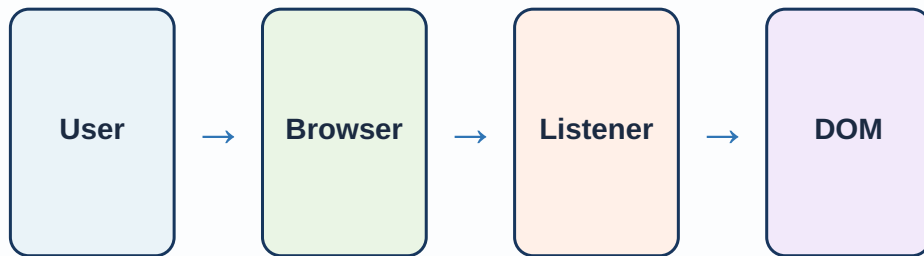
// Change style
title.style.color = "blue";

// Create new element
const p = document.createElement("p");
p.innerText = "Created by JavaScript";
container.appendChild(p);

// Remove element
p.remove();
```

1.4 DOM events: user action triggers JavaScript

Event flow



Events make the webpage interactive.

Input example

```
<input id="nameInput" placeholder="Enter name">
<button id="submitBtn">Submit</button>
<p id="output"></p>

<script>
document.getElementById("submitBtn")
  .addEventListener("click", function () {
    const name = document.getElementById("nameInput").value;
    document.getElementById("output").innerText = "Hello, " + name;
  });
</script>
```

2. JavaScript as a dynamic programming language

What does dynamic mean?

JavaScript is dynamic because many decisions happen while the program is running.

- Variable types are decided at runtime.
- The same variable can store different value types.
- Objects can be changed after creation.
- Functions can be passed like normal values.

One variable, many possible values

let value

10
number

"Hello"
string

true
boolean

{ name: "A" }
object

JavaScript decides the current type based on the value stored.

2.1 Dynamic typing and data types

Same variable, different types

```
let value = 10;
console.log(typeof value); // number

value = "Hello";
console.log(typeof value); // string

value = true;
console.log(typeof value); // boolean
```

This flexibility is useful, but it can also create bugs if values are not validated.

Important data types

Type	Example
Number	10, 25.5
String	"Hello"
Boolean	true, false
Undefined	let x;
Null	null
Object	{ name: "Akshara" }
Array	[10, 20, 30]
Function	function greet() {}

2.2 Type coercion and safe coding

Type coercion examples

"5" + 2

"52"

number becomes
string

"5" - 2

3

string becomes
number

5 == "5"

true

loose comparison

5 === "5"

false

checks value and type

Best practices

- Use `const` by default.
- Use `let` when the value changes.
- Avoid `var` in modern JavaScript.
- Prefer `===` instead of `==`.
- Convert input values explicitly.

```
const marks = Number(input.value);  
if (marks >= 40) {  
  console.log("Pass");  
}
```

3. Object-oriented Programming in JavaScript

What is OOP?

OOP means organizing code using objects.

- **Properties** store data.
- **Methods** perform actions.
- **Classes** create similar objects.

```
const student = {  
  name: "Akshara",  
  marks: 85,  
  study: function () {  
    console.log("Studying");  
  }  
};
```

Class and object idea

class Student

name

marks

getResult()

student1

Akshara, 85

student2

Ravi, 55

student3

Meena, 92

A class is a blueprint. Objects are real instances created from it.

3.1 Class syntax: constructor and methods

Class example

```
class Student {  
  constructor(name, marks) {  
    this.name = name;  
    this.marks = marks;  
  }  
  
  getResult() {  
    return this.marks >= 40 ? "Pass" : "Fail";  
  }  
}  
  
const s1 = new Student("Akshara", 85);  
console.log(s1.getResult());
```

Explanation

- `class` defines the blueprint.
- `constructor()` runs when a new object is created.
- `this.name` stores data in the current object.
- `new Student()` creates an object.

Class syntax is beginner-friendly, but JavaScript still uses prototypes internally.

3.2 Four main OOP concepts

OOP pillars

Encapsulation

Keep data and methods together

Abstraction

Hide internal details

Inheritance

Reuse from parent class

Polymorphism

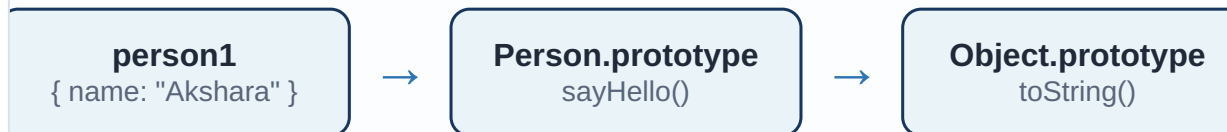
Same method, different behavior

Simple examples

Concept	Easy example
Encapsulation	BankAccount keeps balance and deposit method together
Abstraction	CoffeeMachine.start() hides internal steps
Inheritance	Dog extends Animal
Polymorphism	Dog.sound() and Cat.sound() behave differently

3.3 Prototype chain and this keyword

Prototype chain



If an object does not have a method, JavaScript searches up the prototype chain.

this keyword

`this` usually refers to the object calling the method.

```
const user = {  
  name: "Akshara",  
  greet: function () {  
    console.log("Hello " + this.name);  
  }  
};  
  
user.greet(); // Hello Akshara
```

4. Event loop in JavaScript

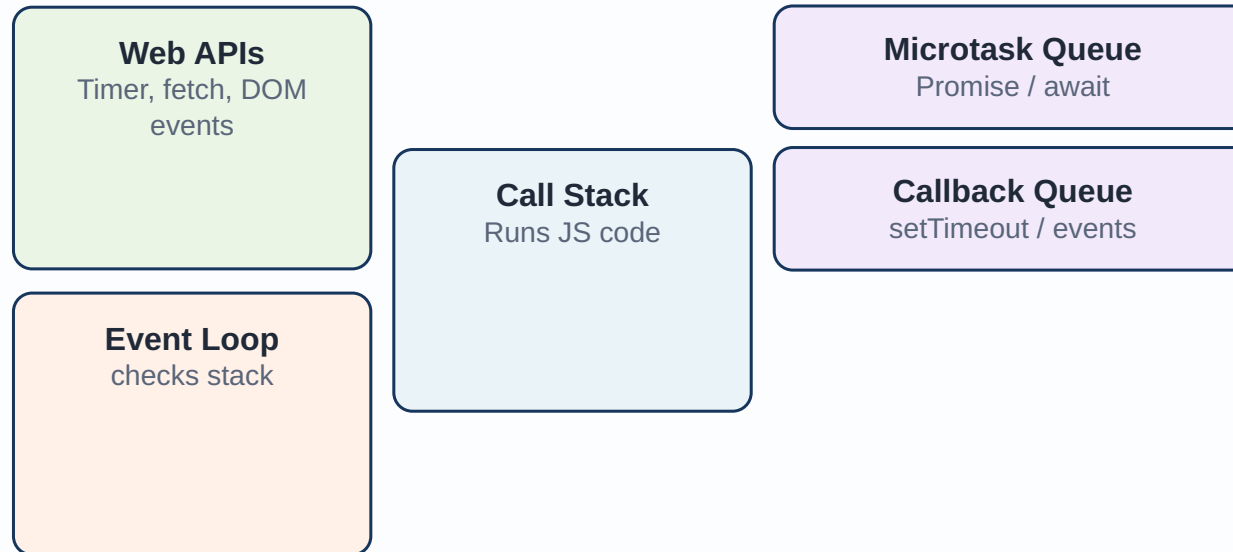
Why do we need it?

JavaScript is single-threaded, but websites handle many things at the same time: button clicks, API calls, timers, promises, animations, and UI updates.

The event loop helps JavaScript run asynchronous callbacks when the call stack is free.

Synchronous code runs first. Async callbacks run later.

Event loop structure



4.1 Main parts of the event loop

Call Stack

Runs normal synchronous JavaScript code line by line.

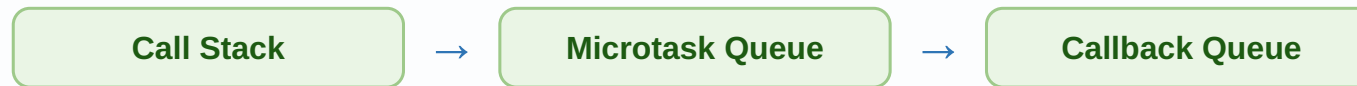
Web APIs

Browser features that handle async work outside the stack.

Queues

Completed async callbacks wait until the stack is empty.

Priority order



Promise callbacks run before **setTimeout** callbacks because microtasks have higher priority.

```
console.log("A");
Promise.resolve().then(() => console.log("B"));
setTimeout(() => console.log("C"), 0);
console.log("D");
```

Output

A
D
B
C

4.2 Step-by-step event loop timeline

Example

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

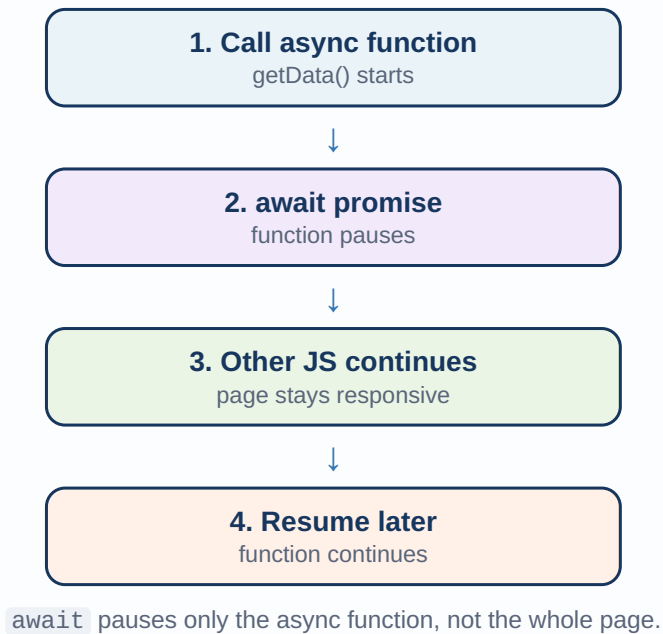
Execution order



Final output: Start → End → Promise → Timeout

4.3 async and await

async / await flow



Example

```
async function getData() {  
  console.log("Before");  
  await Promise.resolve();  
  console.log("After");  
}  
  
getData();  
console.log("Outside");
```

Output: Before → Outside → After

4.4 Fetch API and DOM events with event loop

Fetch example

```
console.log("Start");

fetch("https://api.example.com/users")
  .then(response => response.json())
  .then(data => console.log(data));

console.log("End");
```

The API request starts, JavaScript continues, and the callback runs after the response arrives.

DOM event example

```
document.getElementById("btn")
  .addEventListener("click", function () {
    console.log("Button clicked");
  });
```

The callback waits until the user clicks and the call stack is free.

The event loop keeps the UI responsive while waiting for user actions and async results.

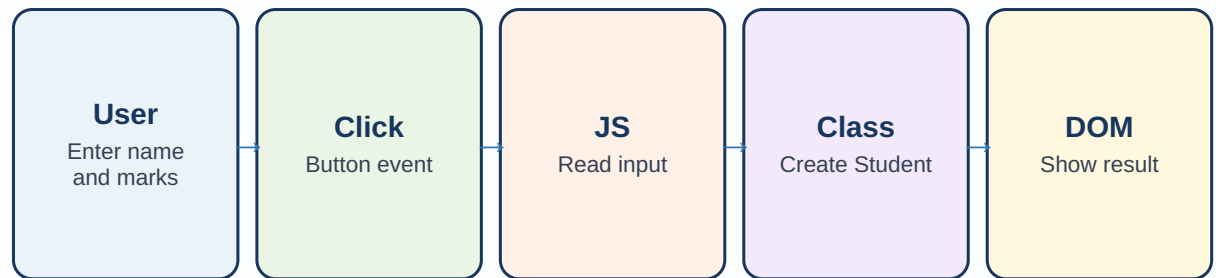
5. Mini project: DOM + Event + OOP

Student result app

This mini project combines the main concepts:

- DOM reads input values.
- Click event starts the logic.
- Class stores student data and behavior.
- DOM shows the final result.

Project flow



5.1 Complete mini project code

HTML

```
<h1>Student Details</h1>

<input type="text" id="studentName" placeholder="Enter student name">
<input type="number" id="studentMarks" placeholder="Enter marks">

<button id="addBtn">Add Student</button>
<p id="result"></p>
```

JavaScript

```
class Student {
  constructor(name, marks) {
    this.name = name;
    this.marks = marks;
  }

  getResult() {
    return this.marks >= 40
      ? this.name + " passed"
      : this.name + " failed";
  }
}

document.getElementById("addBtn")
  .addEventListener("click", function () {
    const name = document.getElementById("studentName").value;
    const marks =
      Number(document.getElementById("studentMarks").value);
    const student = new Student(name, marks);
    document.getElementById("result").innerText = student.getResult();
  });
```

6. Quick revision notes

DOM cheat sheet

Need	Use
Select by id	getElementById()
Select with CSS	querySelector()
Read text	innerText
Read input	value
Handle click	addEventListener()

Core idea cheat sheet

Concept	Remember
Dynamic language	Types are checked at runtime
OOP	Objects group data and behavior
Prototype	Objects can inherit from other objects
Event loop	Async callbacks run when stack is free
Promise	Microtask queue, higher priority

Best beginner practice: write small examples, predict the output, then run and compare.

7. Common interview questions

DOM questions

Question	Simple answer
What is DOM?	Object representation of HTML page
How to read input?	Use element.value
innerText vs innerHTML?	Text only vs HTML content
How to handle click?	addEventListener('click', callback)

Event loop questions

Question	Simple answer
Is JavaScript single-threaded?	Yes, main execution uses one call stack
Promise or setTimeout first?	Promise callback first
What is microtask queue?	Queue for promise callbacks
Why event loop?	To run async tasks without blocking UI

Final summary

JavaScript reads and updates webpages using the DOM. It is dynamic because variable types are decided at runtime. It supports OOP using objects, classes, and prototypes. It handles timers, promises, API calls, and user events using the event loop.